



UiA University
of Agder

Full summary of Discret 2

by

Sander og Erlend

Grimstad, OKT 2024

Contents

1	Chapter 1: Introduction and Concepts	1
1.1	Theory	1
1.1.1	Peano axioms	1
1.1.2	Definition of natrual numbers	1
1.2	Quiz	3
1.2.1	Sets	3
1.2.2	Sets of natrual multiplies	3
1.2.3	Contra position	3
1.2.4	Truth tables	4
1.3	Exams	4
2	Chapter 2: Induction and Recursion	5
2.1	Theory	5
2.1.1	Well-ordering principle	5
2.1.2	Induction	5
2.1.3	Recursion	5
2.1.4	Integer Division	5
2.1.5	Greatest Common Divisor	5
2.1.6	Fundamental Theorem of Arithmetic	6
2.1.7	Strings	6
2.2	Quiz	6
2.2.1	How to calculate the base	6
2.3	Exams	7
3	Chapter 3: Grammars	8
3.1	Theory	8
3.1.1	Languages	8
3.1.2	Grammars	8
3.1.3	Derivation	8
3.1.4	Trees	9
3.1.5	Constructing a syntax tree from grammar	9
3.1.6	Tools	10
3.2	Quiz	10
3.2.1	Derivation of string using EBNF gramamr	10
3.2.2	Ambiguity of the EBNF grammar	10
3.2.3	Translate EBNF grammar into a 4 tuple	10
3.2.4	How to generate strings from a grammar	11
3.3	Exams	11
4	Chapter 4: Languages, Regex and Automata	12
4.1	Theory	12
4.1.1	Languages	12
4.1.2	Regular Expressions	12
4.1.3	Automata	13
4.1.4	NFA and regex	14
4.1.5	Minimize automata	16
4.2	Quiz	17
4.2.1	Regular expression	17
4.2.2	DFA to 4 tuple	17
4.2.3	Minimize DFA	18
4.3	Exams	18
5	Chapter 5: Correct programs	19
5.1	Theory	19
5.1.1	Correctness	19

5.1.2	Temporal logic	19
5.1.3	Abstraction & Substitution	19
5.1.4	Design by Contract	20
5.1.5	Refinement	21
5.2	Quiz	21
5.2.1	Refinement	21
5.2.2	Temporal logic	21
5.3	Exams	21
6	Chapter 6: Relations	22
6.1	Theory	22
6.1.1	Binary Relations	22
6.1.2	Properties of Relations	22
6.1.3	Finite Relations	23
6.1.4	Orders	23
6.1.5	Equivalences	23
6.2	Quiz	23
6.2.1	Properties of the Relation R	23
6.3	Exams	24
7	Chapter 7: Recurrence Relations	25
7.1	Theory	25
7.2	Solving Recurrence Relations	25
7.3	Geometric Progressions	25
7.4	Inspection	25
7.5	Generating Functions	25
7.6	Linear Homogeneous Recurrence Relations of Order 2	25
7.6.1	Case of $r_1 \neq r_2$	25
7.6.2	Case of $r_1 = r_2$	25
7.7	Non-Homogeneous Linear Relations of Order 2	25
7.7.1	Linear Non-Homogeneous Recurrence Relations of Order 1	25
7.8	Quiz	25
7.8.1	Initial value problem	25
7.8.2	Sequence p_n and special recurrence relation	26
7.8.3	Second order recurrence relation satisfied by p_n	26
7.8.4	Recurrence relation with letters	27
7.9	Exams	27
8	Chapter 8: Introduction to Group Theory	28
8.1	Theory	28
8.1.1	Preliminaries on Set Theory	28
8.1.2	Definition	28
8.1.3	Subgroups	28
8.1.4	Examples	28
8.1.5	Homomorphisms and Isomorphisms	28
8.1.6	Finite Groups	28
8.1.7	Cyclic Groups	28
8.2	Quiz	28
8.2.1	Group permutations P	28
8.2.2	Subgroup generated by elements and direct subgroups	28
8.2.3	D_3 dihedral group of symmetries	29
8.3	Exams	29
9	Chapter 9: Cosets and Lagrange's Theorem	30
9.1	Theory	30
9.1.1	Cosets of groups and Lagrange's theorem	30
9.1.2	The Euclidean algorithm	30

9.1.3	B ezout's identity and the extended Euclidean algorithm	30
9.1.4	Multiplication modulo n and the group P_n	30
9.2	Quiz	30
9.2.1	Lagrange's theorem and dihedral groups	30
9.2.2	Coprime of natrual numbers and mode	31
9.3	Exams	31
10	Chapter 10: Applications of Lagrange's Theorem	32
10.1	Theory	32
10.1.1	Euler's Theorem	32
10.1.2	Fermat's Little Theorem	32
10.1.3	The RSA Encryption Scheme	32
10.1.4	Private and Public Keys	33
10.1.5	Modular Exponentiation	33
10.2	Quiz	33
10.2.1	Find valid key pair RSA	33
10.2.2	RSA encryption scheme with public key	34
10.3	Exams	34
11	Chapter 11: Coding Theory	35
11.1	Theory	35
11.1.1	Elements of Coding Theory	35
11.1.2	The Hamming Metric	36
11.1.3	Generator Matrix and Encoding Function	36
11.1.4	Parity Check Matrix and Syndrome	36
11.2	Quiz	37
11.2.1	Hamming distance	37
11.2.2	Noisy binary symmetric channel	37
11.3	Exams	38
12	Chapter 12: Coding Theory and Group Codes	39
12.1	Theory	39
12.1.1	Group Codes: Decoding with Coset Leaders	39
12.2	Quiz	40
12.2.1	Compute syndrome r, error pattern $r + c + e$, encoded message	40
12.2.2	Coding Theory: Minimum Distance and Codewords	41
12.3	Exams	42

1 Chapter 1: Introduction and Concepts

1.1 Theory

1.1.1 Peano axioms

1. 0 is a number.
 $0 \in \mathbb{N}$
2. The successor of any number is a number.
 $\forall n \in \mathbb{N} \rightarrow s(n) \in \mathbb{N}$
3. If a and b are numbers, and if their successors are equal, then a and b are equal.
 $\forall a \forall b (a \in \mathbb{N} \wedge b \in \mathbb{N} \wedge s(a) = s(b) \rightarrow a = b)$
4. 0 is not the successor of any number.
 $\neg \exists n (n \in \mathbb{N} \wedge s(n) = 0)$
5. If S is a set of numbers containing 0, and if the successor of any number in S is contained in S as well, then S contains all the numbers.
 $\forall S (0 \in S \wedge \forall n (n \in S \rightarrow s(n) \in S) \rightarrow \mathbb{N} \subseteq S)$

1.1.2 Definition of natrual numbers

Definitions for natural numbers

•Definition of addition:

$$\bullet 0 + a = a \quad (+.1)$$

$$\bullet s(x) + a = s(x + a) \quad (+.2)$$

•Definition of multiplication:

$$\bullet 0 \cdot a = 0 \quad (\cdot.1)$$

$$\bullet s(x) \cdot a = (x \cdot a) + a \quad (\cdot.2)$$

•Definition of exponentiation:

$$\bullet a^0 = s(0) \quad (^{.1})$$

$$\bullet a^{s(x)} = a^x \cdot a \quad (^{.2})$$

Figure 1: Enter Caption

What is 1+1?

• We only have 0 and s().	$1 = s(0)$	(n.1)
We define 1 to be s(0).	$2 = s(1)$	(n.2)
• Now $1+1 =$ can be	$3 = s(2)$	(n.3)
computed:	$4 = s(3)$	(n.4)
• $1+1 =$	(by n.1)	$5 = s(4)$
• $s(0)+s(0)=$	(by +.2)	$6 = s(5)$
• $s(0+s(0))=$	(by +.1)	$7 = s(6)$
• $s(s(0))=$	(by n.1)	$8 = s(7)$
• $s(1)=$	(by n.2)	$9 = s(8)$
• 2		

Figure 2: Enter Caption

An example would be:

- Let n be a natural number.

$$\begin{aligned}
 n^1 &= n^{s(0)} && \text{by n.1} \\
 &= n^0 * n && \text{by } \wedge.2 \\
 &= s(0) * n && \text{by } \wedge.1 \\
 &= 0 * n + n && \text{by } *.2 \\
 &= 0 + n && \text{by } *.1 \\
 &= n && \text{by } +.1
 \end{aligned}$$

1.2 Quiz

1.2.1 Sets

Consider the set:

$$\{4, 7, \{4, 7\}, \{7\}, \{\{4\}\}, \{\{7\}\}\}.$$

Mark all true statements.

Correct Statements:

- $\{7\} \in S$
- $4 \in S$
- $7 \in S$
- $\{\{4\}\} \subseteq S$
- $\{\{7\}\} \subseteq S$

1.2.2 Sets of natrual multiplies

We use the notation $\{x \cdot n \mid n \in \mathbb{N}\}$ for all natural multiples of x .

Compute the following set:

$$\{4 \cdot n \mid n \in \mathbb{N}\} \cup \{12 \cdot n \mid n \in \mathbb{N}\} = \{4 \cdot n \mid n \in \mathbb{N}\}$$

When we are promptet with $\cup$ we could use the gcd fromula resulting in the number 4.

Compute the following set:

$$\{3 \cdot n \mid n \in \mathbb{N}\} \cap \{9 \cdot n \mid n \in \mathbb{N}\} = \{9 \cdot n \mid n \in \mathbb{N}\}$$

When promptet with $\cap$ we use the lcm formula resulting in the 9.

1.2.3 Contra position

Rules:

$$\neg(A \wedge B) = \neg A \vee \neg B$$

$$\neg(A \vee B) = \neg A \wedge \neg B$$

$$A \implies B = \neg A \vee B$$

$$\neg(\neg p) = p$$

Example:

$$(p \vee \neg q) \implies (s \vee \neg t)$$

$$\neg(s \vee \neg t) \implies \neg(p \vee \neg q)$$

$$\neg s \vee t \implies \neg p \vee q$$

1.2.4 Truth tables

Rules

First Table (AND)

A	B	$A \wedge B$
T	T	T
T	F	F
F	T	F
F	F	F

Second Table (Implication)

A	B	$A \implies B$
T	T	T
T	F	F
F	T	T
F	F	T

Third Table (OR)

A	B	$A \vee B$
T	T	T
T	F	T
F	T	T
F	F	F

Example

p	q	r	$p \wedge q$	$p \vee (p \wedge q)$	$r \implies (p \vee (p \wedge q))$
F	F	F	F	F	T
F	F	T	F	F	T
F	T	F	F	F	T
F	T	T	F	F	T
T	F	F	F	T	T
T	F	T	F	T	T
T	T	F	T	T	T
T	T	T	T	T	T

1.3 Exams

2 Chapter 2: Induction and Recursion

2.1 Theory

2.1.1 Well-ordering principle

Every nonempty subset of the natural numbers contains a smallest element. This principle can turn a property (a set) into an element. Example: Let f be any function from $\mathbb{N}$ to $\mathbb{N}$, then its range has a smallest element. This principle is powerful for infinite sets, as finite sets always have a smallest element

2.1.2 Induction

- $[S(0) \wedge \forall k \geq 0 [S(k) \Rightarrow S(k+1)]] \Rightarrow \forall n \geq 0 [S(n)]$
- We use three steps for inductive proofs:
 - Prove the property initially for 0 (or the start value(s)).
 - Assume the implication condition.
 - Prove the implication consequence.

Example

- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- Prove for 1:

$$\sum_{i=1}^1 i = 1 = \frac{1(1+1)}{2} = \frac{2}{2} = 1$$

- Assume for k :

$$\sum_{i=1}^k i = \frac{k(k+1)}{2}$$

- Prove for $k+1$:

$$\begin{aligned} \sum_{i=1}^{k+1} i &= \left(\sum_{i=1}^k i \right) + (k+1) = \frac{k(k+1)}{2} + (k+1) \\ &= \frac{k(k+1)}{2} + \frac{2(k+1)}{2} = \frac{(k+1)(k+2)}{2} \end{aligned}$$

2.1.3 Recursion

- $[f(0) = v_0 \wedge \forall k \geq 0 [f(k+1) = g(f(k))]] \Rightarrow \forall n \geq 0 \exists v_n [f(n) = v_n]$
- We use two parts for recursive definitions:
 - Define the function initially for 0 (or the start value(s)).
 - Define the function for the successor based on the value of the previous number(s).

2.1.4 Integer Division

calculate bases?

2.1.5 Greatest Common Divisor

How to calculate gcd: gcd(96,684)

$$684/96 = 7 \text{ remainder } 12$$

$$7/96 = 0$$

$$\text{gcd}(96, 684) = 12$$

How to calculate lcm

$$\text{lcm} = \frac{a \cdot b}{\text{gcd}(a, b)}$$

2.1.6 Fundamental Theorem of Arithmetic

2.1.7 Strings

- ε (the empty string) is a string.
- If s is a string and l is a letter from the alphabet A , then $s \circ l$ (appending l at the end of s) is a string.
- There are no other strings.

2.2 Quiz

2.2.1 How to calculate the base

Base 10 to 8

3303_{10} to base 8

$$3303 \div 8 = 412 \quad \text{remainder } 7$$

$$412 \div 8 = 51 \quad \text{remainder } 4$$

$$51 \div 8 = 6 \quad \text{remainder } 3$$

$$6 \div 8 = 0 \quad \text{remainder } 6$$

$$6347_8 = [3303, 412, 51, 6, 0]$$

Base 10 to 26

3303_{10} to base 26

$$3303 \div 26 = 127 \quad \text{remainder } 1$$

$$127 \div 26 = 4 \quad \text{remainder } 23$$

$$4 \div 26 = 0 \quad \text{remainder } 4$$

$$4X1_{26} = [3303, 127, 4, 0]$$

Remember 0-9 is as it is. 10 is A and 11 is B. This means that 1 stays as 1, 23 goes to X and 4 stays as 4.

Base 17 to base 15

First step is to convert base 17 to base 10. This is shown by an example: Here we see that 2 is the second index, 5 is the first index and E is the 0 index.

$$\begin{aligned}25E_{17} &= 2 \times 17^2 + 5 \times 17^1 + E \times 17^0 \\&= 2 \times 289 + 5 \times 17 + 14 \\&= 578 + 85 + 14 \\&= 677_{10}\end{aligned}$$

677_{10} to base 15

$$677 \div 15 = 45 \quad \text{remainder } 2$$

$$45 \div 15 = 3 \quad \text{remainder } 0$$

$$3 \div 15 = 0 \quad \text{remainder } 3$$

$$302_{15} = [677, 45, 3, 0]$$

2.3 Exams

3 Chapter 3: Grammars

3.1 Theory

3.1.1 Languages

3.1.2 Grammars

Grammars example

- Find an EBNF grammar for $a^n b^m c^n$, i.e. a sequence of a's followed by a sequence of b's and then a sequence of c's of the same length as the a's.

$$s \rightarrow a s c \mid B$$

$$B \rightarrow b B \mid \varepsilon$$

Figure 3: Enter Caption

3.1.3 Derivation

Example derivation

- Example EBNF (id is an identifier, num is a number)
 - $E \rightarrow \text{'id'} \mid \text{'num'} \mid E O E \mid \text{'(' } E \text{')'} \mid S E$
 - $O \rightarrow \text{'*'} \mid \text{'+'} \mid \text{'-'} \mid \text{'/'}$
 - $S \rightarrow \text{'+'} \mid \text{'-'} \mid \text{'('}$
- Find a derivation in this grammar for the string $1+2*3$
- A possible derivation is $E \rightarrow EOE \rightarrow EOEEOE \rightarrow 1OEEOE \rightarrow 1+EOE \rightarrow 1+2OE \rightarrow 1+2*E \rightarrow 1+2*3$

Figure 4: Enter Caption

3.1.4 Trees

Solution: Syntax tree for derivation

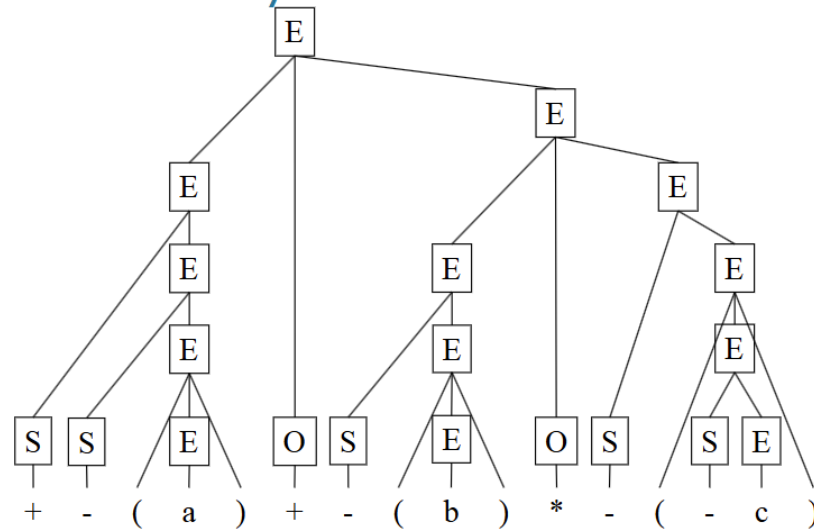
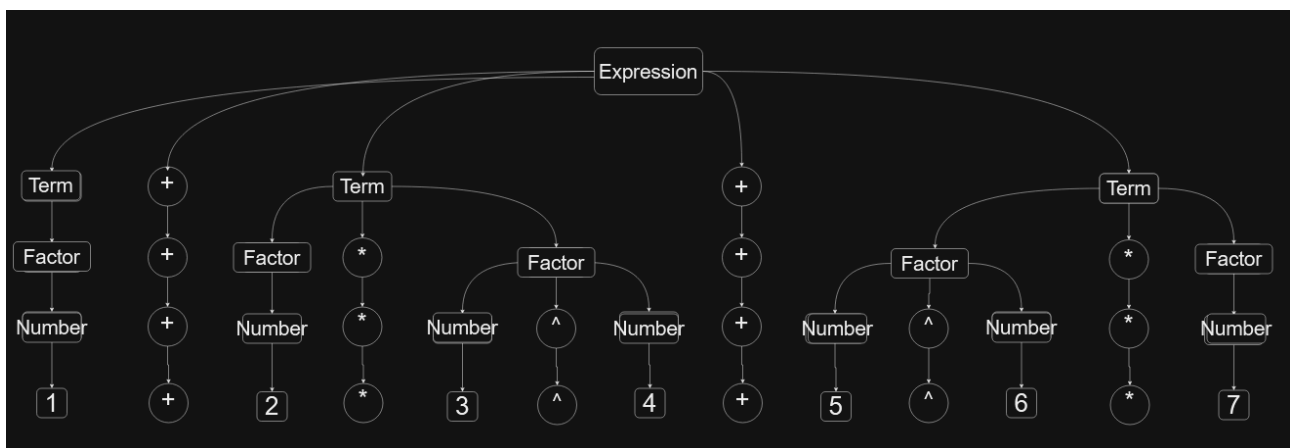


Figure 5: Enter Caption

3.1.5 Constructing a syntax tree from grammar

Construct the syntax tree with the grammar below for the string: $1 + 2 * 3^4 + 5^6 * 7$.

- **expression** = *term* { (+ | -) *term* }
- **term** = *factor* { (* | /) *factor* }
- **factor** = { *number* } *number*

Figure 6: Syntax tree for the string $1 + 2 * 3^4 + 5^6 * 7$.

3.1.6 Tools

3.2 Quiz

3.2.1 Derivation of string using EBNF gramamr

The given EBNF grammar:

$$\begin{aligned} C &\rightarrow H \mid C * C \\ H &\rightarrow D \mid D + H \\ D &\rightarrow i \mid a \mid f \mid d \mid h \mid p \mid \varepsilon \end{aligned}$$

Derivation for the string $i + i * i + h$:

$$\begin{aligned} [C, C * C, H * C, H * H, D + H * D + H, D + D * D + H, \\ D + D * D + D, D + D * D + D, i + D * D + D, i + i * D + D, \\ i + i * i + D, i + i * i + h] \end{aligned}$$

3.2.2 Ambiguity of the EBNF grammar

Ambiguity in grammars occurs when a given grammar allows multiple valid interpretations or different syntax trees for the same input string. This ambiguity makes it unclear how to derive or parse a string uniquely, which is problematic in programming language design and parsing.

If different possible syntax trees can be generated for the same input, the grammar is ambiguous. **Grammar:**

$$\begin{aligned} M &\rightarrow K \mid xM[x] \\ W &\rightarrow d \mid Wed \mid \varepsilon \\ H &\rightarrow F \mid FjH \mid \epsilon \\ F &\rightarrow d \mid Fcd \mid \epsilon \\ K &\rightarrow m \mid mmm \end{aligned}$$

The grammar is ambiguous with the start symbol M	True
The grammar is ambiguous with the start symbol W	False
The grammar is ambiguous with the start symbol H	True
The grammar is ambiguous with the start symbol F	False
The grammar is ambiguous with the start symbol K	False

3.2.3 Translate EBNF grammar into a 4 tuple

Given EBNF Grammar:

$$\begin{aligned} Owl &\rightarrow \text{cup} \{ \text{Hen} \mid \text{bag} \}^+ \mid \text{bus Fox} \\ Fox &\rightarrow \text{Owl fan Hen} \\ Hen &\rightarrow \text{bar} \{ \text{Owl} \} [\text{hat Hen}] \end{aligned}$$

Translated into a Grammar 4-Tuple:

- **Terminals:** {cup, bag, bus, fan, bar, hat}
- **Non-Terminals:** {Owl, Fox, Hen}
- **Start Symbol:** Owl
- **Rules:** Sander hvordan gjør man rules?

3.2.4 How to generate strings from a grammar

You are given the grammar: $(a, b, c, S, A, B, C, S, (S, A), (S, Ac), (S, BC), (A, Aa), (B, b), (C, Cc), (C, c))$

- Terminals: a, b, c
- Non Terminals: S, A, B, C
- Start: S
- Rules: $(S, A), (S, Ac), (S, BC), (A, Aa), (B, b), (C, Cc), (C, c)$

Some of the strings that can be generated are: bc, aac and others

3.3 Exams

4 Chapter 4: Languages, Regex and Automata

4.1 Theory

4.1.1 Languages

4.1.2 Regular Expressions

Regular expressions (regex) are sequences of characters that define a search pattern, primarily used for matching and manipulating text. Regular expressions often feature elements that are beyond regular languages.

•The following operators are used in patterns.

•Operators

- |: alternative
- (): parenthesis for grouping
- ?: optional (0..1)
- *: Kleene closure (0..)
- +: positive closure (1..)
- {n}: exactly n
- {n,}: at least n (n..)
- {,m}: at most m (..m)
- {n,m}: between n and m (n..m)

•Equalities

- $a? = (a|\epsilon)$
- $a+ = aa^*$
- $a^* = a+|\epsilon$
- $a\{n\} = a...a$ (n repetitions)
- $a\{n,\} = a\{n\}a^*$
- $a\{,m\} = (a?)\{m\}$
- $a\{n,m\} = a\{n\}a\{,m-n\}$

•Avoid parentheses: precedence and associativity

- * highest, left
- concatenation next, left
- | lowest, left

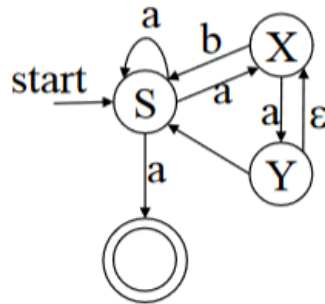
Example:

$$ab | cd^* = (ab) | (c(d^*)) \quad (*|)$$

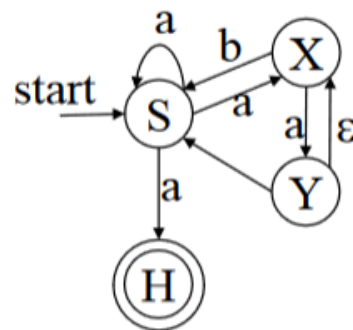
4.1.3 Automata

Example grammar to NFA:

- $S ::= aS \mid aX \mid a$
- $X ::= bS \mid aY$
- $Y ::= bS \mid X$



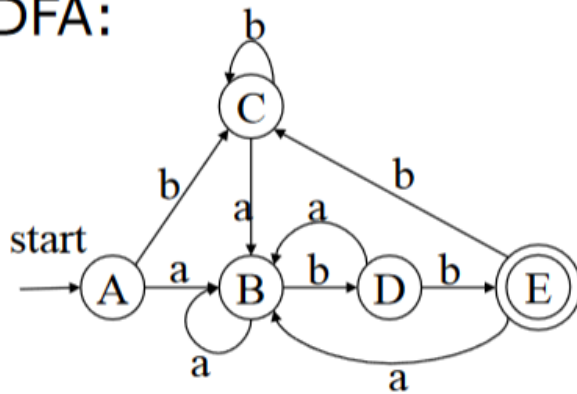
Example NFA to grammar:



- $S ::= aS \mid aX \mid aH$
- $X ::= bS \mid aY$
- $Y ::= bS \mid X$
- $H ::= \epsilon$

Example DFA:

DFA:

Language: $[ab]^*abb$

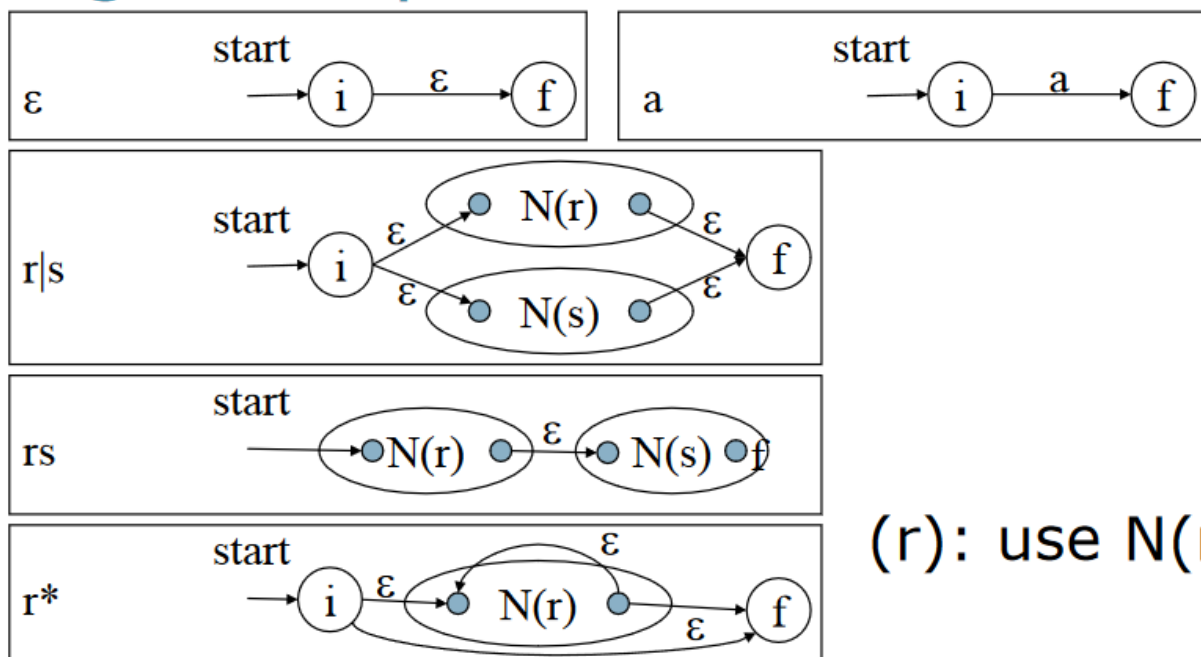
Grammar:

 $A ::= B a b b .$ $B ::= a B \mid b B \mid \epsilon .$

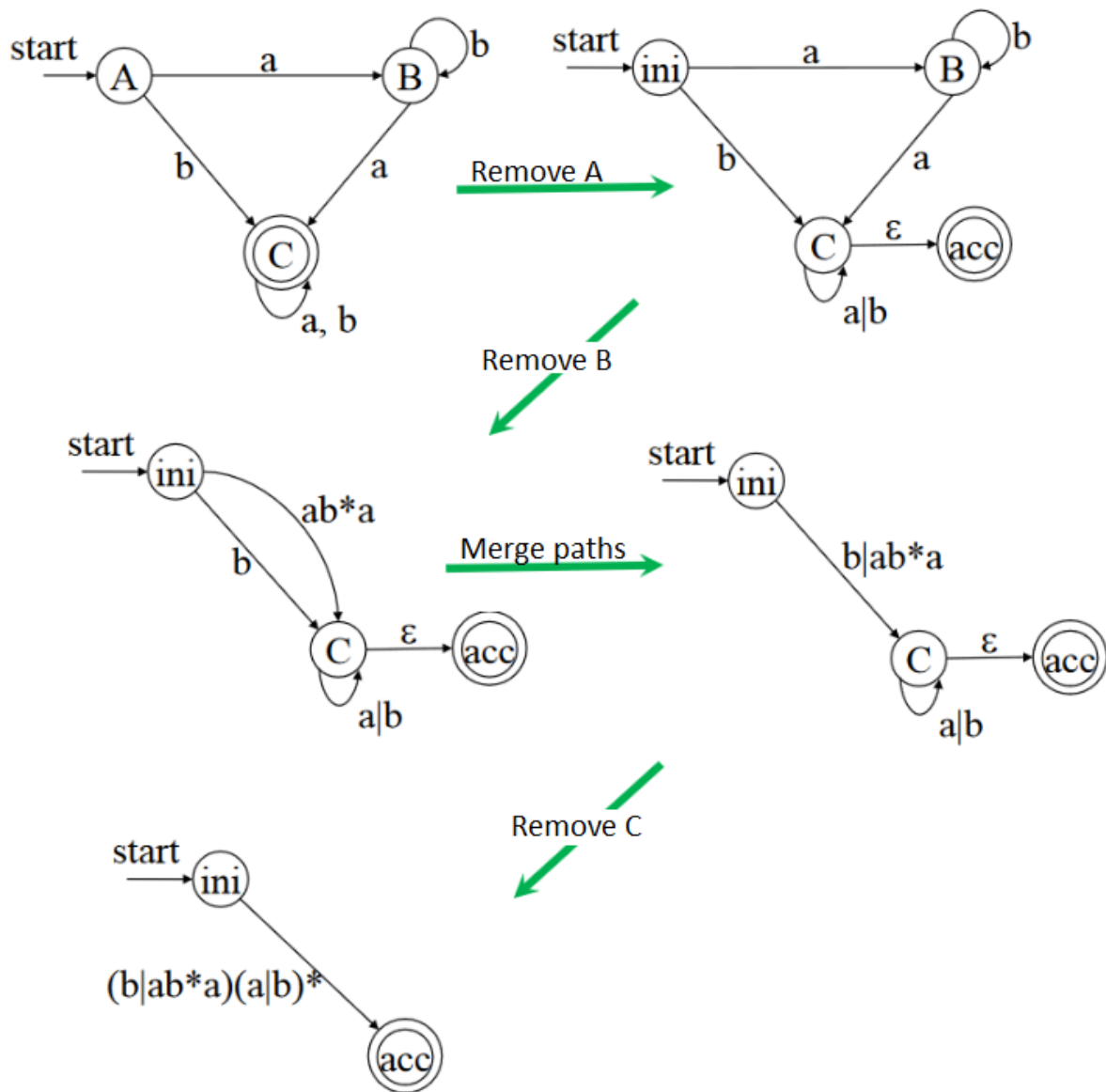
sample inputs: abb, **aabb**, ababb, babb,
abbabb, abbbbabb, abab

4.1.4 NFA and regex

Regex to NFA:

 (r) : use $N(r)$

NFA to Regex



4.1.5 Minimize automata

- DFA minimization stands for converting a given DFA to its equivalent DFA with minimum number of states. The following algorithm will do.
 1. Remove unreachable states.
 2. Divide the set of states into two sets: final states and non-final states. This partition is called P_0 .
 3. Find P_{k+1} recursively by partitioning the sets of P_k . In each set of P_k , separate the states that are distinguishable, thus getting P_{k+1} .
 4. Stop when $P_{k+1} = P_k$ (No change in partition)
 5. The sets of the last P_k are the states of the minimal DFA and have their transitions.
- Two states s_i and s_j are distinguishable in partition P_{k+1} if for any input symbol a , their transitions $T(s_i, a)$ and $T(s_j, a)$ are in different sets in partition P_k .

4.2 Quiz

4.2.1 Regular expression

Consider the following regular expression. $ex = [A-J]??[0-9]^+$ Which of the following regular expressions accepts at least the same words as ?

To do this look at the rules for regular expression, and find others that could make the same letters. Some of them could be:

A-J ?[0-9]*

A-J ?[0-9]^+

A-J *[0-9]*

A-J *[0-9]^+

4.2.2 DFA to 4 tuple

Translate the following DFA to a right-regular grammar.

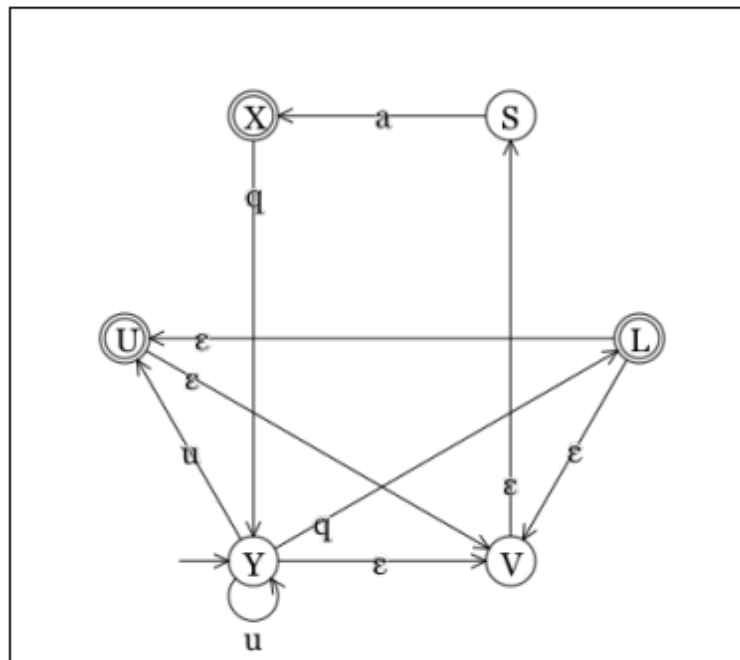


Figure 7: Enter Caption

- Terminals = "a", "q", "u"
- Nonterminals = "X", "S", "L", "V", "Y", "U"
- Start symbol = "Y"
- Rules for L = ["L", ["V"]], ["L", ["U"]], ["L", []]
- Rules for V = ["V", ["S"]]
- Rules for U = ["U", ["V"]], ["U", []]
- Rules for X = ["X", ["q", "Y"]], ["X", []]
- Rules for S = ["S", ["a", "X"]]

4.2.3 Minimize DFA

Follow the rules from 4.1.5.

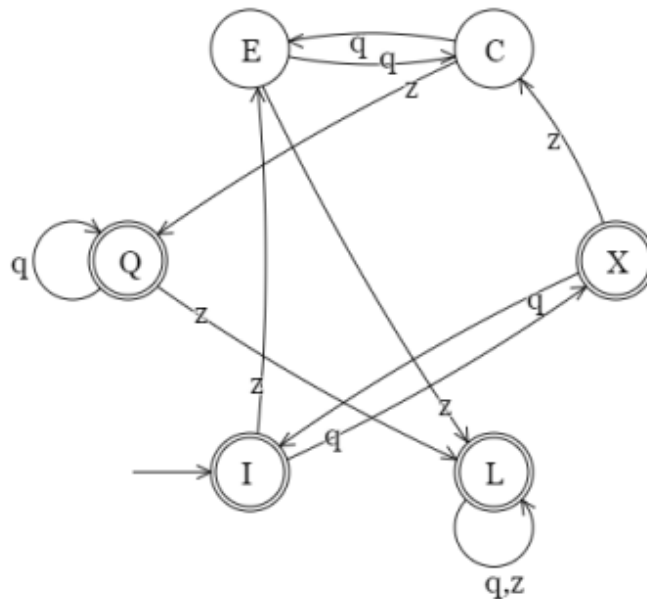


Figure 8: Enter Caption

We start by fining P_0 by deviding the states into non final and final states:

$$P_0 = \{\{E, C\}, \{Q, I, L, X\}\}$$

We check for all states, if a state goes to another subgroup we split it.

I points at a final state and at a non final state so we have to split it

$$P_1 = \{\{E, C\}, \{I\}, \{Q, L, X\}\}$$

Same goes for X

$$P_1 = \{\{E, C\}, \{I\}, \{X\}, \{Q, L\}\}$$

However X and I have the exact same behavior so we can put them togheter in a subset. This gives us the final answer:

$$P_1 = \{\{E, C\}, \{I, X\}, \{Q, L\}\}$$

4.3 Exams

5 Chapter 5: Correct programs

5.1 Theory

5.1.1 Correctness

5.1.2 Temporal logic

Semantic Intuition of LTL

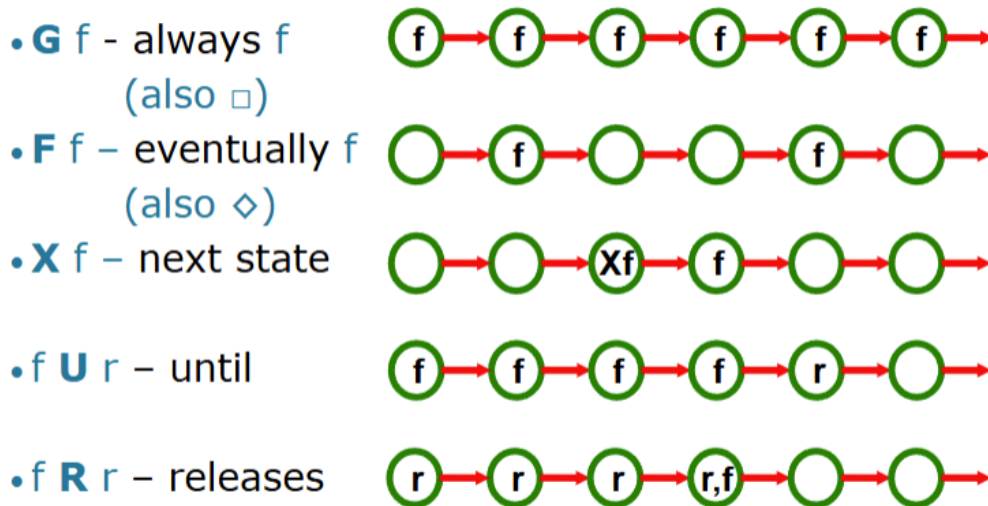


Figure 9: Enter Caption

5.1.3 Abstraction & Substitution

Abstraction & Information Hiding

- **Process abstraction** (hide code)
- **Data abstraction** (hide representation, e.g., `int`, `stack`)
 - Define constructors that create data.
 - Define operators that handle data.
 - Define observers that produce (simple) values.
- **Data format abstraction** (hide low-level transport)

Abstraction by Encapsulation

- Create entities that can be reused and are self-contained (e.g., `modules`, `classes`).
- **Modules** have two parts: interface and specification.
- Enables building programs out of components.
- Avoid attributes in interfaces; use methods (e.g., `get`, `set`) for better flexibility when making changes.

Abstract Data Types (ADTs)

- The declarations of the type and the protocols of the operations on objects of the type [...] are contained in a single syntactic unit.
- The representation of objects of the type is hidden from the program units that use the type.

Typical LTL formulae

- mutual exclusion:
 $\mathbf{G}\neg(\text{critical}_1 \ \& \ \text{critical}_2)$
- it is infinitely often my turn: $\mathbf{GF}\text{myTurn}$
- entering the try region will eventually lead to the critical region:
 $\mathbf{G}(\text{try} \rightarrow \mathbf{F} \text{critical})$
- entering the try region, try holds as long as critical does not hold:
 $\mathbf{G}(\text{try} \rightarrow (\text{critical} \ \mathbf{R} \ \text{try}))$
- after initialisation the system stays initialised: $\mathbf{F} \ \mathbf{G} \ \text{initialised}$

Figure 10: Enter Caption

Example ADT: Peano in Java

```
public interface Peano {
    Peano zero();
    Peano succ(Peano p);
    Peano pred(Peano p);
    Peano add(Peano p1, Peano p2);
}

public class PeanoImpl implements Peano {
    // Implementation details
};
```

5.1.4 Design by Contract

Natural (Peano) in JML

```
public interface Peano {
    //@ ensures \result != null
    public Peano zero();

    //@ requires p != null;
    //@ ensures \result != null && \result != zero()
    public Peano succ(Peano p);

    //@ requires p != null && p != zero();
    //@ ensures \result != null && succ(\result) == p
    public Peano pred(Peano p);

    //@ requires p1 != null && p2 != null;
    //@ ensures \result == (p2==zero()) ? p1 : succ(add(p1,pred(p2)))
    public Peano add(Peano p1, Peano p2);
}
```

Figure 11: Enter Caption

5.1.5 Refinement

Look at quizzes

5.2 Quiz

5.2.1 Refinement

Apply the refinement rule "Rewrite" (see below) to the following specification statement.

Specification statement: $\{ q < d \} \ q, y, d := q', y', d' \mid q = 3 * 1 \text{ and } y' = q + 1$

Refinement rule Rewrite (Rew4)

$v := v' \mid E1 = E2 \text{ and } \text{rel}[E1]$

[=

$v := v' \mid E1 = E2 \text{ and } \text{rel}[E2]$

Fill in the correctly refined specification statement. You can copy parts of the original statement.

$\{ q < d \} \ q, y, d := q', y', d' \mid q = 3 * 1 \text{ and } y' = 3 * 1 + 1$

Figure 12: Enter Caption

5.2.2 Temporal logic

Which temporal logic statement matches the English sentence "Some sunset leads to a continuous sunrise."?

$F(\text{sunset} \Rightarrow G \text{ sunrise})$

Figure 13: Enter Caption

5.3 Exams

6 Chapter 6: Relations

6.1 Theory

6.1.1 Binary Relations

(Binary) Relation

- Any subset of $A \times B$ is called a (binary) relation from A to B .
- Any subset of $A \times A$ is called a (binary) relation on A .
- Examples:
 - $A \times A$ means everything is related, $\emptyset$ means nothing is related
 - $\text{Id}(A) = \{(a, a) \mid a \in A\}$ Identity on A

Figure 14: Enter Caption

Binary relation examples (1)

- $\emptyset$ is the empty binary relation
- $\text{Id}(A) = \{(a, a) \mid a \in A\}$ Identity on A
- $A \times A$ is the complete binary relation
- $\subset, \subseteq, \supset, \supseteq$ are relations on Sets
- $<$ and $\leq$ are relations on Integers
- $>$ and $\geq$ are relations on Integers
- $=$ and $\neq$ are general relations
- Note: $=$ is the same as Id

Figure 15: Enter Caption

6.1.2 Properties of Relations

Reflexive Relations

A relation R on A is reflexive if $(a, a) \in R$ for all $a \in A$.

Example: Consider the set $A = \{1, 2, 3\}$. The relation $R = \{(1, 1), (2, 2), (3, 3)\}$ is reflexive because every element in A is related to itself.

Symmetric Relations

A relation R on A is symmetric if $(a, b) \in R$ implies $(b, a) \in R$ for all $a, b \in A$.

Example: Consider the set $A = \{1, 2\}$. The relation $R = \{(1, 2), (2, 1)\}$ is symmetric because if $(1, 2) \in R$, then $(2, 1) \in R$.

Transitive Relations

A relation R on A is transitive if $(a, b), (b, c) \in R$ implies $(a, c) \in R$ for all $a, b, c \in A$.

Example: Consider the set $A = \{1, 2, 3\}$. The relation $R = \{(1, 2), (2, 3), (1, 3)\}$ is transitive because $(1, 2) \in R$ and $(2, 3) \in R$ imply $(1, 3) \in R$.

Antisymmetric Relations

A relation R on A is antisymmetric if $(a, b), (b, a) \in R$ implies $a = b$ for all $a, b \in A$.

Example: Consider the set $A = \{1, 2\}$. The relation $R = \{(1, 1), (2, 2)\}$ is antisymmetric because no pair $(a, b) \in R$ exists such that $a \neq b$ and $(b, a) \in R$.

Properties of relations

Reflexivity



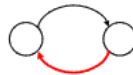
Symmetry



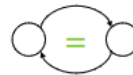
Transitivity



Antisymmetry



or



Prepared by Andreas Pries

1

Figure 16: Enter Caption

6.1.3 Finite Relations

jeg skjønner ikke boolean functions og matriser

6.1.4 Orders

Partial order A relation R is called a partial order if R is reflexive, antisymmetric, and transitive

Totally ordered A partial order R is called a total order if for all a, b in A aRb or bRa

6.1.5 Equivalences

Equivalences relation An equivalence relation R on a set A is a relation that is reflexive, symmetric, and transitive

6.2 Quiz

6.2.1 Properties of the Relation R

Consider the following relation R over integers:

$$xRy \iff (3 + x\%6) = (3 + y\%6)$$

Check the properties of the relation R :

- The relation is reflexive: **True**
- The relation is symmetric: **True**
- The relation is antisymmetric: **False**
- The relation is transitive: **True**

Consider the following relation R over integers:

$$xRy \iff x + y = 56$$

Check the properties of the relation R :

- The relation is reflexive: **False**
- The relation is symmetric: **True**
- The relation is antisymmetric: **False**
- The relation is transitive: **False**

6.3 Exams

7 Chapter 7: Recurrence Relations

Se på handin 7 dokumentet

7.1 Theory

7.2 Solving Recurrence Relations

7.3 Geometric Progressions

7.4 Inspection

7.5 Generating Functions

7.6 Linear Homogeneous Recurrence Relations of Order 2

7.6.1 Case of $r_1 \neq r_2$

Sub-case: Real Roots

Sub-case: Complex Roots

7.6.2 Case of $r_1 = r_2$

7.7 Non-Homogeneous Linear Relations of Order 2

7.7.1 Linear Non-Homogeneous Recurrence Relations of Order 1

7.8 Quiz

7.8.1 Initial value problem

When solving the initial value problem we have to consider one thing. Does the abc formula give us one, two or complex roots

Two roots gives:

$$a_n = c_1 \cdot x_1^n + c_2 \cdot x_2^n$$

One root gives

$$a_n = (c_1 + c_2 \cdot n) \cdot x^n$$

Complex roots:

When the characteristic equation of the recurrence relation has complex roots $r_1 = ke^{i\theta}$ and $r_2 = ke^{-i\theta}$, the general solution can be written using real-valued functions as:

$$a_n = k^n \cdot (c \cdot \cos(n\theta) + d \cdot \sin(n\theta)),$$

where c and d are determined by the initial conditions.

Example complex root:

$$r^2 - \sqrt{3} \cdot r + 1 = 0$$

which has two complex solutions $r_{1,2} = \frac{1}{2} \pm i\frac{\sqrt{3}}{2}$. On polar form, these solutions are $e^{\pm i\pi/6}$. The general solution to the recurrence relation is then

$$a_n = A \cos(n\pi/6) + B \sin(n\pi/6),$$

Example

Find the solution to the initial value problem:

$$\begin{cases} a_n - 4a_{n-1} + 4a_{n-2} = 0, \\ a_0 = 1, \\ a_1 = 10. \end{cases}$$

Step 1: Solve recurrence relation with abc formula. You will end up with one or two roots.

Step 2: Use the formula and the initial values a_0 and a_1 and try to find c_1 and c_2 .

7.8.2 Sequence p_n and special recurrence relation

The following sequence:

$$p_n = \frac{2 \cdot (-1)^n}{7}$$

is a special solution to the recurrence relation:

$$4 \cdot a_n + \frac{a_{n-1}}{2} = (-1)^n.$$

a) Find another solution to the recurrence relation, not equal to p_n .

To solve this task remove $(-1)^n$ from the recurrence relation then solve for a_n . This will give you a fraction. Take $p_n + \text{fraction}$. In this example it would be:

$$a_n = \frac{2 \cdot (-1)^n}{7} + \left(\frac{-1}{8}\right)^n$$

b) Given the initial condition $a_0 = -\frac{5}{7}$, compute a_n for $n = 1$.

Put the values in the recurrence relation or in the new a_n we just found.

7.8.3 Second order recurrence relation satisfied by p_n

Two players, A and B, play a game of chance. Player A has n euros, and Player B has $100 - n$ euros. A random number between 1 and 100 determines who pays whom:

- With probability $\frac{51}{100}$, Player A wins 1 euro from Player B.
- With probability $\frac{49}{100}$, Player A loses 1 euro to Player B.

The probability p_n is the chance that Player A eventually wins the game starting with n euros. If $n = 0$, $p_0 = 0$ since Player A is already broke.

The task is to find a recurrence relation for p_n .

The recurrence relation is derived based on the expected value of p_n :

$$p_n = \frac{49}{100} \cdot p_{n-1} + \frac{51}{100} \cdot p_{n+1}.$$

Rearranging this equation gives:

$$49 \cdot p_{n+1} - 100 \cdot p_n + 51 \cdot p_{n-1} = 0.$$

This is the required 2nd-order recurrence relation, which relates p_n , p_{n-1} , and p_{n+1} , considering the probabilities of winning or losing on each turn.

Task b) Compute the probability of player A winning at the moment the game starts:

The formula for p_{52} is given as:

$$p_{52} = \frac{1}{\left(\frac{51}{49}\right)^{100} - 1} + \frac{1}{\left(\frac{51}{49}\right)^{100} - 1} \cdot \left(\frac{51}{49}\right)^{52}.$$

7.8.4 Recurrence relation with letters

Let A and X be the sets:

$$A = \{a, b, c, d, e\},$$

$$X = \{\times, +, -\}.$$

However, not all words are valid. The following rules apply:

1. A word must start with a letter from the set A .
2. A word must end with a letter from the set A .
3. A word cannot contain two adjacent letters from the set X .

Find a recurrence relation satisfied by a_n :

Us formula where A is length of a , and X is length of X .

$$a_n = A \cdot a_{n-a} + A \cdot X \cdot a_{n-2}$$

This gives us:

$$a_n = 5 \cdot a_{n-1} + 15 \cdot a_{n-2}$$

To find a_1 we find the length of A which is 5. So $a_1 = 5$.

To find a_3 we use:

$$a_3 = \text{length of } A \cdot \text{length of } A + \text{length of } X \cdot \text{length of } A$$

Find the general solution to the recurrence relation satisfied by a_n

How could we do this sander? Special one

7.9 Exams

8 Chapter 8: Introduction to Group Theory

Look at handing 8 for the theory.

8.1 Theory

8.1.1 Preliminaries on Set Theory

8.1.2 Definition

8.1.3 Subgroups

8.1.4 Examples

8.1.5 Homomorphisms and Isomorphisms

8.1.6 Finite Groups

8.1.7 Cyclic Groups

8.2 Quiz

8.2.1 Group permutations P

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 5 & 4 & 3 & 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 3 & 5 & 4 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 4 & 5 & 2 & 3 \end{bmatrix}$$

Inverse permutation

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 3 & 5 & 4 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 5 & 1 & 2 & 4 & 3 \end{bmatrix}$$

8.2.2 Subgroup generated by elements and direct subgroups

In this problem, you are tasked with computing subgroups of $(\mathbb{Z}_n, +_n)$ and direct products of such groups.

a) Find the subgroup generated by the element $0 \in \mathbb{Z}_3$.

The subgroup generated by 0 includes all multiples of 0 under modulo 3:

$$\langle 0 \rangle = \{0 \cdot 0, 0 \cdot 1, 0 \cdot 2\}.$$

Calculating each element:

$$0 \cdot 0 = 0, \quad 0 \cdot 1 = 0, \quad 0 \cdot 2 = 0.$$

Thus, the subgroup is:

$$\langle 0 \rangle = \{0\}.$$

b) Find the subgroup generated by the element $2 \in \mathbb{Z}_4$.

The subgroup generated by 2 includes all multiples of 2 under modulo 4:

$$\langle 2 \rangle = \{2 \cdot 0, 2 \cdot 1, 2 \cdot 2, 2 \cdot 3\}.$$

Calculating each element:

$$2 \cdot 0 \mod 4 = 0, \quad 2 \cdot 1 \mod 4 = 2, \quad 2 \cdot 2 \mod 4 = 0, \quad 2 \cdot 3 \mod 4 = 2.$$

Thus, the subgroup is:

$$\langle 2 \rangle = \{0, 2\}.$$

c) Find the subgroup generated by the element $[0, 2] \in \mathbb{Z}_3 \times \mathbb{Z}_4$.

The subgroup generated by $[0, 2]$ includes all multiples of $[0, 2]$ under the group operation:

$$\langle [0, 2] \rangle = \{[0, 0], [0, 2]\}.$$

Calculating each element:

$$[0, 0] = [0 \cdot 0, 2 \cdot 0],$$

$$[0, 2] = [0 \cdot 1, 2 \cdot 1].$$

Thus, the subgroup is:

$$\langle [0, 2] \rangle = \{[0, 0], [0, 2]\}.$$

We could also combine the answer from a) with the answer from b).

8.2.3 D₃ dihedral group of symmetries

The multiplication table for the dihedral group D_3 (symmetries of the triangle) is:

id	r_1	r_2	t_1	t_2	t_3
r_1	r_2	id	t_2	t_3	t_1
r_2	id	r_1	t_3	t_1	t_2
t_1	t_3	t_2	id	r_2	r_1
t_2	t_1	t_3	r_1	id	r_2
t_3	t_2	t_1	r_2	r_1	id

8.3 Exams

9 Chapter 9: Cosets and Lagrange's Theorem

9.1 Theory

9.1.1 Cosets of groups and Lagrange's theorem

9.1.2 The Euclidean algorithm

9.1.3 Bezout's identity and the extended Euclidean algorithm

9.1.4 Multiplication modulo n and the group P_n

9.2 Quiz

9.2.1 Lagrange's theorem and dihedral groups

Let $G = (\mathbb{Z}_2)^5$, and let $H \subset G$ be the subgroup:

$$H = \{[0, 0, 0, 0, 0], [0, 0, 1, 1, 0], [0, 1, 0, 1, 1], [0, 1, 1, 0, 1]\}.$$

What is the index of H in G ?

To compute the index $[G : H]$, we use the formula:

$$[G : H] = \frac{|G|}{|H|},$$

where $|G|$ is the order (size) of the group G , and $|H|$ is the order of the subgroup H .

The group $G = (\mathbb{Z}_2)^5$ consists of all 5-tuples of elements from $\mathbb{Z}_2$, which means G has 2^5 elements:

$$|G| = 2^5 = 32.$$

The subgroup H is explicitly given with 4 elements:

$$|H| = 4.$$

Substituting into the formula:

$$[G : H] = \frac{|G|}{|H|} = \frac{32}{4} = 8.$$

Thus, the index of H in G is:

$$[G : H] = 8.$$

The index of H in G is 8.

Part b) Let D_4 be the dihedral group of order 8, representing the symmetries of the unit square centered at the origin. The subgroup $\langle \sigma \rangle$ is generated by σ , which represents a rotation by -90° around the origin. Compute the index of the subgroup $\langle \sigma \rangle$ in D_4 .

To compute $[D_4 : \langle \sigma \rangle]$, we use the formula:

$$[D_4 : \langle \sigma \rangle] = \frac{|D_4|}{|\langle \sigma \rangle|},$$

where $|D_4|$ is the order of the group D_4 , and $|\langle \sigma \rangle|$ is the order of the subgroup generated by σ .

The dihedral group D_4 has order 8, representing the 4 rotations and 4 reflections of the square:

$$|D_4| = 8.$$

The subgroup $\langle \sigma \rangle$ consists of all powers of σ , which generates the cyclic subgroup of rotations. Since σ corresponds to a -90° rotation, the subgroup $\langle \sigma \rangle$ contains:

$$\langle \sigma \rangle = \{\text{id}, \sigma, \sigma^2, \sigma^3\},$$

where $\sigma^4 = \text{id}$. Thus, $|\langle \sigma \rangle| = 4$.

Substituting into the formula:

$$[D_4 : \langle \sigma \rangle] = \frac{|D_4|}{|\langle \sigma \rangle|} = \frac{8}{4} = 2.$$

The index of $\langle \sigma \rangle$ in D_4 is:

$$[D_4 : \langle \sigma \rangle] = 2.$$

9.2.2 Coprime of natrual numbers and mode

To compute modular arithmetic, we use the operation:

$$a \cdot b \mod n,$$

or in the given notation:

$$a \bullet_n b,$$

where a and b are the operands, and n is the modulus.

Example 1: $7 \bullet_9 8$

1. Multiply 7 and 8:

$$7 \cdot 8 = 56.$$

2. Compute $56 \mod 9$ by dividing 56 by 9 and finding the remainder:

$$56 \div 9 = 6 \text{ remainder } 2.$$

Thus:

$$7 \bullet_9 8 = 2.$$

Example 2: $8 \bullet_9 5$

1. Multiply 8 and 5:

$$8 \cdot 5 = 40.$$

2. Compute $40 \mod 9$ by dividing 40 by 9 and finding the remainder:

$$40 \div 9 = 4 \text{ remainder } 4.$$

Thus:

$$8 \bullet_9 5 = 4.$$

9.3 Exams

10 Chapter 10: Applications of Lagrange's Theorem

10.1 Theory

Theorem 1.1 (Lagrange's Theorem). Let G be a finite group, and let $H \subseteq G$ be a subgroup. Then:

$$|G| = |H| \cdot [G : H].$$

This implies that the order (size) of any subgroup H of a group G divides the order of G . Moreover, if $a \in G$, the order of the cyclic subgroup $\langle a \rangle$ generated by a also divides $|G|$.

Corollary: For any $a \in G$, $a^{|G|} = e$, where e is the identity element of G .

10.1.1 Euler's Theorem

Euler's theorem states that for the multiplicative group of integers modulo n :

$$P_n = \{k \in \mathbb{Z} \mid 1 \leq k < n, \gcd(k, n) = 1\},$$

with the group operation defined by integer multiplication modulo n . If a and n are coprime, then:

$$a^{\phi(n)} \equiv 1 \pmod{n},$$

where $\phi(n)$ is Euler's totient function, defined as the number of integers less than n that are coprime to n :

$$\phi(n) = |P_n|.$$

Properties of $\phi(n)$:

- If p is a prime:

$$\phi(p) = p - 1.$$

- If p and q are primes:

$$\phi(pq) = \phi(p) \cdot \phi(q) = (p - 1)(q - 1).$$

10.1.2 Fermat's Little Theorem

Fermat's Little Theorem is a special case of Euler's theorem when n is prime. It states that if p is a prime and a is an integer such that $1 \leq a < p$, then:

$$a^{p-1} \equiv 1 \pmod{p}.$$

Example: To test if 527 is a prime, compute $2^{526} \pmod{527}$. If the result is not 1, then 527 is not a prime.

10.1.3 The RSA Encryption Scheme

The RSA encryption scheme involves the following steps: 1. Choose two prime numbers p and q , and compute $n = pq$. 2. Compute Euler's totient function:

$$\phi(n) = (p - 1)(q - 1).$$

3. Choose two integers x and y such that:

$$x \cdot y \equiv 1 \pmod{\phi(n)}.$$

Encryption: Given a message m , where $0 \leq m < n$:

$$M = m^x \pmod{n}.$$

Decryption: Given the encrypted message M :

$$m = M^y \pmod{n}.$$

Reason: By Euler's theorem:

$$(m^x)^y = m^{xy} \equiv m \pmod{n}.$$

10.1.4 Private and Public Keys

In RSA, the public key is (n, x) , while the private key is (n, y) . Messages are encrypted using the public key and decrypted using the private key. This ensures secure communication.

10.1.5 Modular Exponentiation

Modular exponentiation is a technique for efficiently computing powers modulo n . For integers a , b , and n , compute:

$$a^b \pmod{n}.$$

Intermediate results are reduced modulo n to ensure computations remain manageable:

$$37^3 \pmod{13} \equiv 11^3 \pmod{13} \equiv 5.$$

This method is computationally efficient and is widely used in cryptographic algorithms like RSA.

10.2 Quiz

10.2.1 Find valid key pair RSA

Given the parameters $x = 36661763$, $y = 31000091$, $p = 5783$, $q = 6553$, and $n = 37895999$, we verify whether these values form a valid RSA key pair using the following steps:

The modulus n in RSA is the product of two prime numbers p and q . Compute:

$$n = p \cdot q = 5783 \cdot 6553 = 37895999.$$

Thus, the given n is valid.

Using the formula for Euler's totient function for $n = p \cdot q$:

$$\phi(n) = (p - 1)(q - 1).$$

Substitute $p = 5783$ and $q = 6553$:

$$\phi(n) = (5783 - 1)(6553 - 1) = 5782 \cdot 6552 = 37883664.$$

The public key x and the private key y must satisfy:

$$x \cdot y \equiv 1 \pmod{\phi(n)}.$$

Substitute $x = 36661763$, $y = 31000091$, and $\phi(n) = 37883664$. Compute:

$$x \cdot y = 36661763 \cdot 31000091 = 1136231469980953.$$

Now, compute:

$$1136231469980953 \pmod{37883664} = 1.$$

Since the result is 1, the keys x and y satisfy the modular inverse relationship.

The given parameters form a valid RSA key pair. Therefore, the index of the subgroup and the RSA pair is valid, and the correct answer is:

$$\boxed{8}.$$

10.2.2 RSA encryption scheme with public key

Bob's public key is given as $n = 3139$ and $x = 883$. To find Bob's private key y , we follow these steps:

Using the list of prime numbers provided, we factor $n = 3139$:

$$n = 3139 = 37 \cdot 83,$$

where $p = 37$ and $q = 83$.

Using the formula for $\phi(n)$:

$$\phi(n) = (p - 1)(q - 1).$$

Substitute $p = 37$ and $q = 83$:

$$\phi(n) = (37 - 1)(83 - 1) = 36 \cdot 82 = 2952.$$

The private key y satisfies:

$$x \cdot y \equiv 1 \pmod{\phi(n)}.$$

We solve for y using the extended Euclidean algorithm for $x = 883$ and $\phi(n) = 2952$. The result is:

$$y = 1387.$$

Bob's private key is: $y = 1387$

Alice encrypts a message using $M = 3121$, and Bob's public key $(n, x) = (3139, 883)$. To decrypt the message, we compute:

$$m = M^y \pmod{n}.$$

Substitute $M = 3121$, $y = 1387$, and $n = 3139$. Using modular exponentiation:

$$m = 3121^{1387} \pmod{3139}.$$

Efficient modular exponentiation gives:

$$m = 1769.$$

The unencrypted (plain text) message is: $m = 1769$

10.3 Exams

11 Chapter 11: Coding Theory

11.1 Theory

11.1.1 Elements of Coding Theory

Calculating the Probability of No Errors

When transmitting a message $c \in \mathbb{Z}_2^k$ (k is the number of bits, i.e., k copies of $(\mathbb{Z}_2, +)$), the probability of **no errors** being transmitted is given by:

$$P_{\text{no errors}} = (1 - p_0)^k,$$

where p_0 is the probability of an incorrect transmission.

Example: Assume we transmit the message $c = 10110 \in \mathbb{Z}_2^5$. If the probability of an incorrect transmission is $p_0 = 0.05$, the probability of the message arriving with no errors is:

$$P_{\text{no errors}} = (1 - 0.05)^5 = 0.95^5 \approx 0.77.$$

Calculating the Probability of Receiving a Specific String

In coding theory, we assume that the transmission of each bit is independent of the others. The probability of receiving a specific modified string $r \in \mathbb{Z}_2^k$, given the original string $c \in \mathbb{Z}_2^k$, can be calculated as:

$$P_{\text{receiving } r} = (p_0)^{H(c,r)} \cdot (1 - p_0)^{k-H(c,r)},$$

where $H(c, r)$ is the Hamming distance between c and r (i.e., the number of differing bits). Alternatively, let a be the number of errors (the Hamming distance $H(c, r)$). Then:

$$P_{\text{receiving } r} = p_0^a \cdot (1 - p_0)^{k-a}.$$

Example: If we transmit $c = 10110 \in \mathbb{Z}_2^5$ and receive $r = 00110 \in \mathbb{Z}_2^5$, with $p_0 = 0.05$, the probability of receiving exactly r is:

$$P_{\text{receiving } r} = (0.05)(0.95)^2(0.05)(0.95) = 0.002.$$

Calculating the Probability of Exactly n Errors

To calculate the probability of exactly n errors occurring in a message of length k , we use the formula:

$$P_{\text{exactly } n \text{ errors}} = \binom{k}{n} \cdot p_0^n \cdot (1 - p_0)^{k-n},$$

where $\binom{k}{n}$ is the binomial coefficient.

Example: The probability of receiving a transmission with exactly 2 errors when transmitting a string of length 5 with $p_0 = 0.05$ is:

$$P_{2 \text{ errors}} = \binom{5}{2} \cdot (0.05)^2 \cdot (0.95)^3 = 0.021.$$

Calculating the Probability of At Most 1 Error

The probability of at most 1 error is the sum of the probabilities of no errors and exactly 1 error. For a message $c = 10110$, the probability is:

$$P_{\text{at most 1 error}} = (1 - p_0)^5 + \binom{5}{1} \cdot p_0 \cdot (1 - p_0)^4.$$

Example: If $p_0 = 0.05$, then:

$$P_{\text{at most 1 error}} = (0.95)^5 + \binom{5}{1} \cdot (0.05) \cdot (0.95)^4 = 0.977.$$

11.1.2 The Hamming Metric

The Hamming distance between two bit strings is calculated by performing a bitwise addition of the two strings (modulo 2) and then counting the number of 1 bits in the resulting string. For two strings c and r , this is equivalent to:

$$\text{Hamming Distance}(c, r) = \text{wt}(r + c),$$

where $\text{wt}(r + c)$ denotes the weight of $r + c$, i.e., the number of 1 bits in the resulting string.

Property: The Hamming weight satisfies the inequality:

$$\text{wt}(c + r) \leq \text{wt}(c) + \text{wt}(r).$$

Transmission and Errors: If a message c is transmitted, and a different message $r = E(c)$ is received, the transmission can result in up to:

$$k = \text{Hamming_Distance}(r, c) - 1$$

errors, where k is the maximum number of errors introduced during the transmission.

11.1.3 Generator Matrix and Encoding Function

We define a generator matrix G as:

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix},$$

where $G = [I_3 \mid A]$, with I_3 being the 3×3 identity matrix.

Using this generator matrix, we can define an encoding function $E : \mathbb{Z}_2^3 \rightarrow \mathbb{Z}_2^6$ as:

$$E(w) = wG,$$

where all operations are performed modulo 2 (i.e., $1 + 1 = 0$). This function maps a 3-bit input w to a 6-bit codeword.

Example: For $w = [1, 1, 0] \in \mathbb{Z}_2^3$, the codeword is:

$$E(110) = [110]G = [110101].$$

Codewords: The set of all codewords generated by G is:

$$C = \{000000, 100110, 010011, 001101, 110101, 101011, 011110, 111000\} \subset \mathbb{Z}_2^6.$$

11.1.4 Parity Check Matrix and Syndrome

To verify if a given string is a codeword, we use the parity check matrix H , defined as:

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix},$$

where $H = [A^T \mid I_3]$ and A^T is the transpose of A .

The parity check matrix can identify whether a string $r \in \mathbb{Z}_2^6$ is a codeword by calculating the syndrome:

$$\text{Syndrome: } s = H \cdot r^T.$$

If $s = [0, 0, 0]$, then r is a codeword. Otherwise, r is not a codeword.

Example: Consider $r = [1, 1, 0, 1, 1, 0] \in \mathbb{Z}_2^6$. The syndrome is calculated as:

$$H \cdot r^T = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

Since $s = [0, 1, 1] \neq [0, 0, 0]$, r is not a codeword.

11.2 Quiz

11.2.1 Hamming distance

We are tasked with computing the minimal ($l_{\min}$) and maximal ($l_{\max}$) Hamming distance between any two bitstrings in the set:

$$\{001010, 110010, 110000, 010001\}.$$

The Hamming distance $H(c, r)$ between two bitstrings c and r is the number of positions at which the corresponding bits differ. For two bitstrings c and r of length k , the Hamming distance is calculated as:

$$H(c, r) = \text{Number of differing bits between } c \text{ and } r.$$

We compute the Hamming distance for all pairs of bitstrings in the set.

$$\begin{aligned} H(001010, 110010) &= 2 && \text{(differs in the 1st and 2nd bits),} \\ H(001010, 110000) &= 3 && \text{(differs in the 1st, 2nd, and 6th bits),} \\ H(001010, 010001) &= 4 && \text{(differs in the 1st, 2nd, 5th, and 6th bits),} \\ H(110010, 110000) &= 1 && \text{(differs in the 6th bit),} \\ H(110010, 010001) &= 4 && \text{(differs in the 1st, 3rd, 5th, and 6th bits),} \\ H(110000, 010001) &= 3 && \text{(differs in the 1st, 3rd, and 5th bits).} \end{aligned}$$

From the computed distances, we identify:

$$\begin{aligned} l_{\min} &= 1 && \text{(from } H(110010, 110000)), \\ l_{\max} &= 4 && \text{(from } H(001010, 010001) \text{ and } H(110010, 010001)). \end{aligned}$$

11.2.2 Noisy binary symmetric channel

The probability of making an error when transmitting a single bit is $p_0 = 0.097$. The transmitted bitstring is $w = 0000101$.

The probability of no errors is given by:

$$P_{\text{no errors}} = (1 - p_0)^k,$$

where k is the length of the bitstring. Here, $k = 7$:

$$P_{\text{no errors}} = (1 - 0.097)^7 = 0.903^7 \approx 0.490.$$

Answer to Part a: The probability of no errors is:

$$\boxed{0.490}.$$

Part b: Probability of Making at Most 2 Errors Transmitting w The probability of making at most 2 errors is the sum of the probabilities of exactly 0, 1, or 2 errors. For exactly n errors, the probability is:

$$P_{\text{exactly } n \text{ errors}} = \binom{k}{n} \cdot p_0^n \cdot (1 - p_0)^{k-n}.$$

Calculate each term:

$$P_{0 \text{ errors}} = \binom{7}{0} \cdot (0.097)^0 \cdot (0.903)^7 = (1) \cdot (0.903)^7 \approx 0.490.$$

$$P_{1 \text{ error}} = \binom{7}{1} \cdot (0.097)^1 \cdot (0.903)^6 = 7 \cdot (0.097) \cdot (0.903)^6 \approx 0.332.$$

$$P_{2 \text{ errors}} = \binom{7}{2} \cdot (0.097)^2 \cdot (0.903)^5 = 21 \cdot (0.097)^2 \cdot (0.903)^5 \approx 0.154.$$

Summing these probabilities:

$$P_{\text{at most 2 errors}} = P_{0 \text{ errors}} + P_{1 \text{ error}} + P_{2 \text{ errors}} \approx 0.490 + 0.332 + 0.154 = 0.976.$$

Answer to Part b: The probability of at most 2 errors is:

$$\boxed{0.976}.$$

Part c: Probability of Transmitting 0000101 and Receiving 0000000 The probability of receiving a specific bitstring r , given w , is:

$$P_{\text{receiving } r} = p_0^a \cdot (1 - p_0)^{k-a},$$

where $a = H(w, r)$ is the Hamming distance between w and r .

The Hamming distance $H(0000101, 0000000)$ is 2 (the 5th and 7th bits differ). Thus:

$$P_{\text{receiving } r} = p_0^2 \cdot (1 - p_0)^5 = (0.097)^2 \cdot (0.903)^5 \approx 0.006.$$

Answer to Part c: The probability of receiving $r = 0000000$ is:

$$\boxed{0.006}.$$

11.3 Exams

12 Chapter 12: Coding Theory and Group Codes

12.1 Theory

12.1.1 Group Codes: Decoding with Coset Leaders

Let $E : \mathbb{Z}_2^m \rightarrow \mathbb{Z}_2^n$ be an encoding function. If $C = E(\mathbb{Z}_2^m)$ is a subgroup of $\mathbb{Z}_2^n$, then C is called a *group code*.

Example: Consider the encoding function $E : \mathbb{Z}_2^2 \rightarrow \mathbb{Z}_2^6$ defined as:

$$E(00) = 000000, \quad E(01) = 010101, \quad E(10) = 101010, \quad E(11) = 111111.$$

The resulting code group is:

$$C = \{000000, 010101, 101010, 111111\} \subset \mathbb{Z}_2^6,$$

which is a subgroup of $\mathbb{Z}_2^6$. The minimum distance between codewords in C is the minimum weight of the nonzero elements in C .

If $E : \mathbb{Z}_2^m \rightarrow \mathbb{Z}_2^n$ is a group homomorphism, then $C = E(\mathbb{Z}_2^m)$ is a subgroup of $\mathbb{Z}_2^n$. Furthermore, for a parity-check matrix H :

$$C = \{c \in \mathbb{Z}_2^n \mid H \cdot c^T = 0\}.$$

Decoding with a Group Code and Coset Leaders

For a group code C generated from a generator matrix G , consider:

$$C = \{000000, 100110, 010011, 001101, 110101, 101011, 011110, 111000\}.$$

This is the first row of the decoding table. To complete the table, we add cosets by flipping individual bits of each codeword in C (starting with bit one, then two, then so on):

Row 1: 000000, 100110, 010011, 001101, 110101, 101011, 011110, 111000,
 Row 2: 100000, 000110, 110011, 101101, 010101, 001011, 111110, 011000,
 Row 3: 010000, 110110, 000011, 011101, 100101, 111011, 001110, 101000,
 Row 4: 001000, 101110, 011011, 000101, 111101, 100011, 010110, 110000,
 Row 5: 000100, 100010, 010111, 001001, 110001, 101111, 011010, 111100,
 Row 6: 000010, 100100, 010001, 001111, 110111, 101001, 011100, 111010,
 Row 7: 000001, 100111, 010010, 001100, 110100, 101010, 011111, 111001,
 Row 8: 010100, 110010, 000111, 011001, 100101, 111111, 001010, 101100.

To decode a received message r , locate r in the table and select the codeword at the top of the same column.

Example: For $r = 101001$, locate it in the table. It appears under the column corresponding to $c = 101011$. Thus, the decoded codeword is 101011, which corresponds to the original message 101 (before encoding).

Cosets and Syndromes: Let C be a group code with parity-check matrix H . Two received words r_1 and r_2 are in the same coset of C if and only if:

$$H \cdot r_1^T = H \cdot r_2^T.$$

Decoding with Syndromes:

To decode using syndromes, extend the decoding table by adding the syndromes (the syndrome is the same for all words in the same row) as a new column:

Syndrome Row 1: 000, 000000, 100110, 010011, 001101, 110101, 101011, 011110, 111000,
 Syndrome Row 2: 110, 100000, 000110, 110011, 101101, 010101, 001011, 111110, 011000,
 Syndrome Row 3: 011, 010000, 110110, 000011, 011101, 100101, 111011, 001110, 101000,
 Syndrome Row 4: 101, 001000, 101110, 011011, 000101, 111101, 100011, 010110, 110000,
 Syndrome Row 5: 100, 000100, 100010, 010111, 001001, 110001, 101111, 011010, 111100,
 Syndrome Row 6: 010, 000010, 100100, 010001, 001111, 110111, 101001, 011100, 111010,
 Syndrome Row 7: 001, 000001, 100111, 010010, 001100, 110100, 101010, 011111, 111001,
 Syndrome Row 8: 111, 010100, 110010, 000111, 011001, 100101, 111111, 001010, 101100.

To decode a received word r :

1. Compute the syndrome $H \cdot r^T$.
2. Locate the corresponding coset leader x in the decoding table.
3. Calculate the codeword as $c = r + x$.

Example: For $r = 110110$, compute the syndrome:

$$H \cdot r^T = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

Locate the coset leader for syndrome 011, which is $x = 010000$. The codeword is:

$$c = r + x = 110110 + 010000 = 100110,$$

corresponding to the original message 100.

Detect Errors with generator: The number of errors a generator is able to detect depends on the minimum hamming distance between two pairs of codewords generated by the G matrix. Since this is the same as just calculating the minimum weight of all non-zero codewords we can just calculate the weight of each such string to find the number of errors the generator is capable of detecting. (errors its capable of detecting = $\min(\text{hamming_distance}(\text{every: } a, b \in G)) - 1$)

12.2 Quiz

12.2.1 Compute syndrome r , error pattern $r + c + e$, encoded message

We use the generator matrix:

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

to define the encoding function:

$$E : (\mathbb{Z}/2)^4 \rightarrow (\mathbb{Z}/2)^8$$

by the formula:

$$E(w) = w \cdot G.$$

An encoded message $c = E(w)$ is transmitted over a noisy channel and received as:

$$r = 01000010.$$

Question a: Compute the Syndrome of $r = 1001$

The syndrome is calculated using:

$$s = H \cdot r^T$$

where H is the parity-check matrix. Given $r = 1001$, and assuming the parity-check matrix H corresponds to the generator matrix G (from the provided theory), we compute:

$$H \cdot r^T = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Compute the syndrome:

$$s = H \cdot r^T = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Answer: The syndrome is 1001.

Question b: Compute the Error Pattern ϵ

The error pattern ϵ is calculated such that:

$$r = c + \epsilon,$$

where r is the received message and c is the transmitted codeword. From the syndrome, we identify the position of the error and set the corresponding bit in ϵ to 1. In this case, the error is in the second position:

$$\epsilon = 00100000.$$

Answer: The error pattern is $\epsilon = 00100000$.

Question c: Compute the Encoded Message $c = 0110010$

To compute the encoded message c , we use the generator matrix G to encode w as:

$$E(w) = w \cdot G.$$

Given $c = 0110010$, we verify c by ensuring:

$$H \cdot c^T = 0,$$

indicating it is a valid codeword. The encoding function confirms this result.

Answer: The encoded message is 0110010.

12.2.2 Coding Theory: Minimum Distance and Codewords

We use the encoding function:

$$E : (\mathbb{Z}/2)^3 \rightarrow (\mathbb{Z}/2)^5$$

defined as:

$$E(w) = w \cdot G,$$

where the generator matrix G is:

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

Question a: Compute the Minimum Distance

The minimum distance $\ell_{\min}$ between any two codewords in the code is the smallest Hamming weight of any nonzero codeword. The Hamming weight is defined as the number of 1's in a codeword.

To compute $\ell_{\min}$, we consider all possible codewords generated by G :

$$C = \{00000, 10011, 01001, 00101, 11010, 10110, 01100, 11111\}.$$

The nonzero codewords are:

$$10011, 01001, 00101, 11010, 10110, 01100, 11111.$$

The Hamming weights of these codewords are:

$$\text{wt}(10011) = 3, \quad \text{wt}(01001) = 2, \quad \text{wt}(00101) = 2, \quad \text{wt}(11010) = 3, \quad \text{wt}(10110) = 3, \quad \text{wt}(01100) = 2, \quad \text{wt}(11111) = 5.$$

The smallest nonzero Hamming weight is:

$$\ell_{\min} = 2.$$

Answer: The minimum distance is:

$$\ell_{\min} = 2.$$

Question b: Find a Non-Zero Codeword

A non-zero codeword is any codeword in C that is not the all-zero codeword 00000. For example, using $w = 101$:

$$E(101) = [101] \cdot G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

Thus:

$$E(101) = 10011.$$

Answer: A non-zero codeword is:

$$c = 10011.$$

Question c: Find a Bitstring x Not in the Code

To find a bitstring $x \in (\mathbb{Z}/2)^5$ that is not a codeword, we check for any bitstring whose syndrome is non-zero. For example, consider $x = 10001$. Verify:

$$H \cdot x^T \neq 0,$$

where H is the parity-check matrix. Since x does not satisfy $H \cdot x^T = 0$, it is not a codeword.

Answer: A bitstring not in the code is:

$$x = 10001.$$

12.3 Exams